

Optimizing Android-Local 3D Computational Fluid Dynamics Using Normal-Driven Geodesic Polygon Sweep Geometry

Christopher Harding

July 2, 2026

Abstract

This document explains how a normal-driven geodesic polygon sweep can optimize local three-dimensional computational fluid dynamics on Android-class hardware. The method converts a heavy three-dimensional body-geometry problem into a compressed swept-volume representation using sectional polygons, arclength resampling, swept boundary triangles, triangle normals, wake-likelihood weights, and pressure-force diagnostics. Instead of attempting workstation-style body-fitted CFD, the method uses geometry to identify where limited mobile compute should be spent: near corners, rear-facing surfaces, pressure-sensitive regions, and the separated wake. The result is a surface-aware CFD acceleration strategy suitable for Android, Pydroid, and other constrained local computing environments.

1 Core Optimization Idea

The goal is not to make Android perform brute-force workstation CFD. The goal is to make the geometry tell the solver where computation matters most.

A workstation-style CFD pipeline often attempts to resolve the body, boundary layer, wake, and pressure field with a large body-fitted mesh. On Android, this is too expensive in memory and runtime. The proposed method replaces that approach with a compact geometric attention system:

OBJ geometry $\rightarrow$ sectional polygons $\rightarrow$ geodesic loops $\rightarrow$ swept triangles $\rightarrow$ normal-morphed shell $\rightarrow$ grid mask, v (1)

The central morphing formula is

$$\widetilde{\mathbf{X}}(s, \theta) = \mathbf{X}(s, \theta) + \alpha(s, \theta)\mathbf{n}(s, \theta), \quad (2)$$

where $\mathbf{X}(s, \theta)$ is the original swept surface, $\mathbf{n}(s, \theta)$ is the local outward surface normal, and $\alpha(s, \theta)$ is a geometry, wake, or pressure-driven morphing amplitude.

In discrete triangle form, this becomes

$$\widetilde{\mathbf{q}}_j = \mathbf{q}_j + \alpha_j \mathbf{N}_j, \quad (3)$$

where $\mathbf{q}_j$ is a polygon vertex and $\mathbf{N}_j$ is a vertex normal obtained from nearby triangle normals.

This gives Android a compact way to define near-body shells, wake regions, and pressure-sampling surfaces without using a full body-fitted mesh.

2 Why Android CFD Needs This

A three-dimensional incompressible CFD solver usually stores fields such as

$$u, \quad v, \quad w, \quad p, \quad \nu_t, \quad \text{mask}, \quad W, \quad C, \quad (4)$$

where u, v, w are velocity components, p is pressure, ν_t is modeled eddy viscosity, W is wake likelihood, and C is corner likelihood.

For a grid of size

$$N_x \times N_y \times N_z, \quad (5)$$

the memory cost scales as

$$O(N_x N_y N_z). \quad (6)$$

This is the main problem on Android. Even moderate three-dimensional grids can consume significant memory. Therefore, the geometry representation must remain small, and the solver must avoid unnecessary refinement.

The swept-triangle representation stores only

$$\{\mathbf{r}_i, \mathbf{n}_i, A_i, C_i, W_i\}_{i=1}^{N_T}, \quad (7)$$

where $\mathbf{r}_i$ is triangle centroid, $\mathbf{n}_i$ is normal, A_i is area, C_i is corner weight, and W_i is wake weight. This geometry cost scales as

$$O(N_T), \quad (8)$$

which is much cheaper than storing a full body-fitted volumetric mesh.

3 Sectional Polygon Compression

Let the body be sliced into streamwise sections located at

$$x_0, x_1, \dots, x_{K-1}. \quad (9)$$

At each station, define a closed polygon

$$P_k = \{(y_{j,k}, z_{j,k})\}_{j=0}^{N-1}. \quad (10)$$

The corresponding three-dimensional vertex is

$$\mathbf{q}_{j,k} = \begin{bmatrix} x_k \\ y_{j,k} \\ z_{j,k} \end{bmatrix}. \quad (11)$$

The body volume can be estimated by sectional integration:

$$V \approx \sum_{k=0}^{K-2} \frac{A_k + A_{k+1}}{2} (x_{k+1} - x_k), \quad (12)$$

where A_k is the area of section P_k .

This is computationally efficient because the full body is reduced to a sequence of two-dimensional polygons. Android does not need to remesh the entire body with tetrahedra or prisms.

4 Geodesic Arclength Resampling

To connect neighboring polygons cleanly, each closed section is resampled by arclength.

Let the two-dimensional polygon boundary be

$$\mathbf{r}_m = \begin{bmatrix} y_m \\ z_m \end{bmatrix}, \quad m = 0, \dots, M-1. \quad (13)$$

The edge length is

$$\ell_m = \|\mathbf{r}_{m+1} - \mathbf{r}_m\|, \quad \mathbf{r}_M = \mathbf{r}_0. \quad (14)$$

The perimeter is

$$\mathcal{L} = \sum_{m=0}^{M-1} \ell_m. \quad (15)$$

A uniform arclength point is

$$\sigma_j = \frac{j}{N} \mathcal{L}, \quad j = 0, \dots, N-1. \quad (16)$$

After interpolation along the boundary, every section has the same number of points. This allows fast, consistent swept-triangle construction.

5 Swept Boundary Triangles

Between two neighboring sections k and $k+1$, define

$$\mathbf{a}_j = \mathbf{q}_{j,k}, \quad (17)$$

$$\mathbf{b}_j = \mathbf{q}_{j+1,k}, \quad (18)$$

$$\mathbf{c}_j = \mathbf{q}_{j+1,k+1}, \quad (19)$$

$$\mathbf{d}_j = \mathbf{q}_{j,k+1}. \quad (20)$$

Each quadrilateral strip becomes two triangles:

$$T_{j,k}^{(1)} = (\mathbf{a}_j, \mathbf{d}_j, \mathbf{c}_j), \quad (21)$$

$$T_{j,k}^{(2)} = (\mathbf{a}_j, \mathbf{c}_j, \mathbf{b}_j). \quad (22)$$

For triangle

$$T_i = (\mathbf{p}_{i0}, \mathbf{p}_{i1}, \mathbf{p}_{i2}), \quad (23)$$

the centroid is

$$\mathbf{r}_i = \frac{\mathbf{p}_{i0} + \mathbf{p}_{i1} + \mathbf{p}_{i2}}{3}. \quad (24)$$

The unnormalized normal is

$$\mathbf{m}_i = (\mathbf{p}_{i1} - \mathbf{p}_{i0}) \times (\mathbf{p}_{i2} - \mathbf{p}_{i0}). \quad (25)$$

The triangle area is

$$A_i = \frac{1}{2} \|\mathbf{m}_i\|. \quad (26)$$

The unit normal is

$$\mathbf{n}_i = \frac{\mathbf{m}_i}{\|\mathbf{m}_i\|}. \quad (27)$$

The swept triangles form a surface-aware replacement for a full body-fitted geometry.

6 Normal-Driven Morphing Shell

The optimization comes from using the triangle normals to build a nearby shell. For each vertex, collect adjacent triangles

$$\mathcal{A}(j) = \{i : T_i \text{ touches vertex } j\}. \quad (28)$$

Define a weighted vertex normal:

$$\mathbf{N}_j = \frac{\sum_{i \in \mathcal{A}(j)} w_i \mathbf{n}_i}{\left\| \sum_{i \in \mathcal{A}(j)} w_i \mathbf{n}_i \right\|}. \quad (29)$$

A simple area-based weight is

$$w_i = A_i. \quad (30)$$

A CFD-aware weight is

$$w_i = A_i \left(1 + \lambda_c C_i + \lambda_w W_i + \lambda_p \frac{|p_i - p_\infty|}{q_\infty} \right), \quad (31)$$

where C_i is corner weight, W_i is wake likelihood, p_i is pressure sampled near the triangle, and

$$q_\infty = \frac{1}{2} \rho U_\infty^2 \quad (32)$$

is freestream dynamic pressure.

The morphed vertex is

$$\tilde{\mathbf{q}}_j = \mathbf{q}_j + \alpha_j \mathbf{N}_j. \quad (33)$$

The morphing amplitude may be chosen as

$$\alpha_j = \epsilon L (a C_j + b W_j + c |N_{jx}|), \quad (34)$$

where L is body length, C_j is corner intensity, W_j is wake intensity, and $|N_{jx}|$ emphasizes front and rear pressure-facing regions.

7 Swept Shell Volume

For small normal displacements, the additional shell volume is approximately

$$\Delta V \approx \sum_i \alpha_i A_i. \quad (35)$$

Therefore,

$$V_{\text{shell}} \approx V_{\text{body}} + \sum_i \alpha_i A_i. \quad (36)$$

This is important for Android because it gives a cheap scalar estimate of the near-body diagnostic volume. The solver does not need to create a heavy 3D mesh around the surface.

8 Grid-Clustering Optimization

Android should not use a uniformly dense three-dimensional grid unless necessary. Instead, use the morphed shell to define a grid-density function:

$$\rho_g(\mathbf{x}) = 1 + \beta_b B(\mathbf{x}) + \beta_w W(\mathbf{x}) + \beta_c C(\mathbf{x}), \quad (37)$$

where $B(\mathbf{x})$ is boundary-shell strength, $W(\mathbf{x})$ is wake strength, and $C(\mathbf{x})$ is corner strength. A hyperbolic-tangent clustering axis may be generated using density bumps of the form

$$\rho(\xi) = 1 + \sum_m a_m \operatorname{sech}^2 \left(\frac{\xi - \xi_m}{h_m} \right). \quad (38)$$

Here ξ may represent x , y , or z . The centers ξ_m are chosen from important geometric regions:

- front face,
- rear base,
- slant region,
- roof,
- ground clearance,
- side edges,
- near wake.

The normal-morphed shell improves this by providing automatic geometry-aware clustering targets.

9 Wake-Seed Optimization

Wake computation is expensive. The normal-guided method defines wake seeds from surface geometry.

A practical wake seed formula is

$$W_i = b_r R_i + b_n \max(n_{ix}, 0) + b_c C_i + b_b B_i, \quad (39)$$

where

- R_i is rear-region weight,
- $\max(n_{ix}, 0)$ measures downstream-facing normal direction,
- C_i is corner strength,
- B_i is rear-base or cap strength.

Only the strongest wake seeds are kept:

$$\mathcal{S}_M = \text{TopM}(W_i). \quad (40)$$

Thus the active wake cost is capped:

$$M \ll N_T. \quad (41)$$

This is a major Android optimization. Instead of spawning wake diagnostics from all triangles, the solver only uses the strongest separation-relevant regions.

10 Recursive Wake Refinement

A bounded recursive wake hierarchy may be generated from the strongest wake seeds. Let an eddy center be

$$\mathbf{c}_\ell = \begin{bmatrix} x_\ell \\ y_\ell \\ z_\ell \end{bmatrix}. \quad (42)$$

A child eddy center may be placed using Euler rotation in the cross-plane:

$$y_{\ell+1} + iz_{\ell+1} = y_\ell + iz_\ell + r_\ell e^{i\theta_\ell}. \quad (43)$$

Using Euler's identity,

$$e^{i\theta} = \cos \theta + i \sin \theta. \quad (44)$$

The radius and amplitude decrease recursively:

$$r_{\ell+1} = \gamma_r r_\ell, \quad (45)$$

$$A_{\ell+1} = \gamma_A A_\ell, \quad (46)$$

with

$$0 < \gamma_r < 1, \quad 0 < \gamma_A < 1. \quad (47)$$

This creates a bounded wake-likelihood refinement, not a DNS turbulence replacement. It is useful because Android can cap the maximum number of eddies:

$$N_{\text{eddy}} \leq N_{\text{max}}. \quad (48)$$

11 Pressure-Force Optimization

The pressure force on the triangulated swept surface is

$$\mathbf{F}_p = - \sum_i p_i A_i \mathbf{n}_i. \quad (49)$$

Drag, side force, and lift are

$$D_p = \mathbf{F}_p \cdot \mathbf{e}_x, \quad (50)$$

$$S_p = \mathbf{F}_p \cdot \mathbf{e}_y, \quad (51)$$

$$L_p = \mathbf{F}_p \cdot \mathbf{e}_z. \quad (52)$$

The nondimensional coefficients are

$$C_D = \frac{D_p}{q_\infty A_{\text{ref}}}, \quad (53)$$

$$C_S = \frac{S_p}{q_\infty A_{\text{ref}}}, \quad (54)$$

$$C_L = \frac{L_p}{q_\infty A_{\text{ref}}}. \quad (55)$$

This is Android-efficient because the solver only samples pressure at triangle centroids:

$$p_i = p(\mathbf{r}_i). \quad (56)$$

The morphed surface gives a sensitivity estimate:

$$\Delta \mathbf{F}_p = - \sum_i \left(\tilde{p}_i \tilde{A}_i \tilde{\mathbf{n}}_i - p_i A_i \mathbf{n}_i \right). \quad (57)$$

Therefore,

$$\Delta C_D = \frac{\Delta \mathbf{F}_p \cdot \mathbf{e}_x}{q_\infty A_{\text{ref}}}. \quad (58)$$

This lets the Android solver report not only drag, but also a geometry-induced drag sensitivity estimate.

12 Importance Field

The normal-morphed geometry creates an importance field:

$$I_i = aC_i + bW_i + c|n_{ix}| + d\frac{|p_i - p_\infty|}{q_\infty}. \quad (59)$$

This triangle importance can be mapped to the grid:

$$I(\mathbf{x}) = \max_i \left[I_i \exp \left(-\frac{\|\mathbf{x} - \mathbf{r}_i\|^2}{2\sigma_i^2} \right) \right]. \quad (60)$$

Then the solver can decide where to spend computation:

$$I(\mathbf{x}) > I_{\min} \quad \Rightarrow \quad \text{high-detail treatment.} \quad (61)$$

$$I(\mathbf{x}) \leq I_{\min} \quad \Rightarrow \quad \text{low-cost treatment.} \quad (62)$$

This gives Android a geometric attention mechanism.

13 Android Runtime Tiers

The method naturally supports multiple local runtime tiers.

13.1 Fast Tier

$$K = 80, \quad (63)$$

$$N = 32, \quad (64)$$

$$M = 24. \quad (65)$$

This tier is useful for quick diagnostics and debugging.

13.2 Balanced Tier

$$K = 120, \quad (66)$$

$$N = 48, \quad (67)$$

$$M = 36 \text{ to } 64. \quad (68)$$

This tier is suitable for practical Android CFD runs.

13.3 High Tier

$$K = 160, \quad (69)$$

$$N = 64, \quad (70)$$

$$M = 80 \text{ to } 120. \quad (71)$$

This tier provides stronger geometry and wake resolution while remaining much cheaper than full body-fitted CFD.

The approximate number of side triangles is

$$N_T \approx 2(K - 1)N. \quad (72)$$

For example, with $K = 160$ and $N = 64$,

$$N_T \approx 2(159)(64) = 20352. \quad (73)$$

This is manageable on Android compared with a dense volumetric mesh.

14 Optimized Android CFD Pipeline

The complete optimized pipeline is

$\text{OBJ} \rightarrow \text{sections} \rightarrow \text{geodesic loops} \rightarrow \text{swept triangles} \rightarrow \text{normal shell} \rightarrow \text{importance grid} \rightarrow \text{projection CFD} \rightarrow C_L$	(74)
--	------

In algorithmic form:

1. Import the OBJ body geometry.
2. Slice the body into streamwise polygon sections.
3. Resample each polygon by closed arclength.
4. Connect adjacent loops into swept triangles.
5. Compute triangle centroid $\mathbf{r}_i$, area A_i , and normal $\mathbf{n}_i$.
6. Compute corner weights C_i and rear weights R_i .
7. Compute wake seed W_i .
8. Compute morphing amplitude α_i .
9. Build the normal-morphed shell.
10. Map shell, corner, and wake weights to the CFD grid.
11. Cluster the grid using geometry-aware importance.
12. Run the incompressible projection CFD update.
13. Sample pressure at swept-triangle centroids.
14. Compute pressure drag and lift.
15. Compute morphed-surface force sensitivity.

15 Flow Solver Remains Required

The morphing swept-volume method does not replace the governing equations. The incompressible continuity equation remains

$$\nabla \cdot \mathbf{u} = 0. \quad (75)$$

The momentum equation remains

$$\frac{\partial \mathbf{u}}{\partial t} + (\mathbf{u} \cdot \nabla) \mathbf{u} = -\frac{1}{\rho} \nabla p + \nabla \cdot [(\nu + \nu_t) (\nabla \mathbf{u} + \nabla \mathbf{u}^T)]. \quad (76)$$

The normal-driven geometry helps the solver decide where to place effort, but it does not eliminate the need for pressure projection, turbulence modeling, boundary conditions, grid convergence, or validation.

16 What the Method Optimizes

The method optimizes Android-local 3D CFD in the following ways:

Optimization Area	Effect
Geometry compression	Replaces heavy body-fitted meshing with sectional polygons and swept triangles.
Boundary treatment	Creates a near-body normal shell for masks, forcing zones, and diagnostics.
Grid placement	Uses geometry and wake importance to cluster cells only where needed.
Wake modeling cost	Keeps only the strongest wake seeds instead of activating the whole surface.
Pressure-force integration	Uses triangle centroids, areas, and normals for cheap pressure drag estimates.
Memory control	Stores compact geometric metadata instead of a full surface-volume mesh.
Runtime scaling	Allows fast, balanced, and high-detail Android modes.
Uncertainty estimation	Uses morphed-surface force differences to estimate geometry-induced drag sensitivity.

17 Limitations

The method should be treated as an optimization layer, not as proof of physical accuracy. It still requires:

- grid-convergence studies,
- time-step sensitivity studies,
- validation against trusted CFD solvers,
- comparison with published Ahmed-body data,

- turbulence-model calibration,
- uncertainty estimates,
- careful monitoring of self-intersection in the morphed shell.

At high Reynolds number, Android-local CFD cannot be considered true DNS unless the grid resolves all relevant turbulent scales. The practical use case is therefore reduced-order, RANS-like, DDES-like, or diagnostic CFD.

18 Conclusion

The normal-driven geodesic polygon sweep optimizes Android-local 3D CFD by turning the geometry into an active computational guide.

The key formula is

$$\boxed{\widetilde{\mathbf{X}} = \mathbf{X} + \alpha \mathbf{n}}. \quad (77)$$

This produces a compact normal-morphed shell whose volume correction is

$$\boxed{\Delta V \approx \sum_i \alpha_i A_i}. \quad (78)$$

Pressure force remains

$$\boxed{\mathbf{F}_p = - \sum_i p_i A_i \mathbf{n}_i}. \quad (79)$$

The morphed-surface sensitivity is

$$\boxed{\Delta \mathbf{F}_p = - \sum_i \left(\tilde{p}_i \tilde{A}_i \tilde{\mathbf{n}}_i - p_i A_i \mathbf{n}_i \right)}. \quad (80)$$

Thus, the Android optimization is not brute force. It is selective computation:

less memory+fewer wake sources+smarter grid clustering+cheaper force integration+better diagnostic uncertainty (81)

The method allows a local Android device to perform meaningful three-dimensional CFD-style experimentation by letting the swept geometry identify where the solver should spend its limited computational budget.